AMRITA VISHWA VIDYAPEETHAM

19ECE204

Digital Electronics and Systems

III Semester

B. Tech. (ECE & CSE)

Amrita School of Engineering, Bengaluru

Number Representation and Arithmetic Circuits

Overview

- Number Representation
- Unsigned Arithmetic
- Signed Arithmetic
- Arithmetic Overflow
- Fast Adders

Number Representation

- Types of Number Representation (Positional Representation)
  - Decimal - Base 10
  - Binary - Base 2
  - Octal - Base 8
  - Hexadecimal - Base 16
- Not limited to this

Number Representation- Positional Number System

- Lets consider decimal number system
  - Possible different numbers 0-9 - (0 - {N-1}, N is base)
  - Each digit represents a multiple of power of 10 (Base)
  - Example: $(8547)_{10}$ - $8x10^3 + 5x10^2 + 4x10^1 + 7x10^0$
  - This is what we call as one's, ten's, hundred's place etc..
- Similarly all the positional number system can be represented

Number Representation - Conversions

- Conversion from one representation to another is possible
- Binary to Decimal Conversion

$$V = b_{n-1} \times 2^{n-1} + \cdots + b_2 \times 2^2 + b_1 \times 2^1 + b_0$$

  - V = Final decimal value
  - b - binary digits (b_{n-1} is MSB and b_0 is LSB)
- Decimal to Binary Conversion
  - Successive division of decimal number by 2
  - Take the reminder from bottom up as the result

Number Representation - Conversions

- Octal to Binary and Vice versa
  - Octal to Binary is just as straightforward; each octal digit is simply replaced by three bits that denote the same value. (Hint: Use 4-2-1 code)
  - Binary to Octal is by grouping binary numbers as group of three, then write the corresponding group value in octal. (Hint: Use 4-2-1 code)

Number Representation - Conversions

- Hexadecimal to Binary and Vice Versa
  - Each hexadecimal digit is simply replaced by four bits that denote the same value. (Hint: Use 8-4-2-1 code)
  - Binary to Hexadecimal is by grouping binary numbers as group of four, then write the corresponding group value in hexadecimal. (Hint: Use 8-4-2-1 code)
- In computers binary is dominant, but octal and hexadecimal is used as short hand notation for binary

Number Representation - Examples

Convert $(857)_{10}$

			Remainder	
$857 \div 2$	=	428	1	LSB
$428 \div 2$	=	214	0	
$214 \div 2$	=	107	0	
$107 \div 2$	=	53	1	
$53 \div 2$	=	26	1	
$26 \div 2$	=	13	0	
$13 \div 2$	=	6	1	
$6 \div 2$	=	3	0	
$3 \div 2$	=	1	1	
$1 \div 2$	=	0	1	MSB

Result is $(1101011001)_2$

Number Representation - Examples

$$\underbrace{101}_{5}\quad\underbrace{011}_{3}\quad\underbrace{010}_{2}\quad\underbrace{111}_{7}$$

$$(10101010111)_2 = (5327)_8$$

$$\underbrace{1010}_{A}\quad\underbrace{1111}_{F}\quad\underbrace{0010}_{2}\quad\underbrace{0101}_{5}$$

$$(1OI0111100100101)_2 = (AF25)_{16}$$

Number Representation

Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Unsigned Arithmetic

- Numbers that are positive only are called unsigned, and numbers that can also be negative are called signed.

- We will deal with binary arithmetic in this course

- Unsigned Binary Addition and Subtraction

  - Binary addition is performed in the same way as decimal addition except that the values of individual digits can be only 0 or 1

Unsigned Arithmetic - Addition

- Single Digit Addition, all possible combinations

$$\begin{array}{r} 0 \\ +\,0 \\ \hline 0\;0 \end{array} \qquad \begin{array}{r} 0 \\ +\,1 \\ \hline 0\;1 \end{array} \qquad \begin{array}{r} 1 \\ +\,0 \\ \hline 0\;1 \end{array} \qquad \begin{array}{r} 1 \\ +\,1 \\ \hline 1\;0 \end{array}$$

$$\begin{array}{r} x \\ +\,y \\ \hline c\;s \end{array}$$

Carry ← c s → Sum

Can we get a truth table for one bit unsigned addition ??

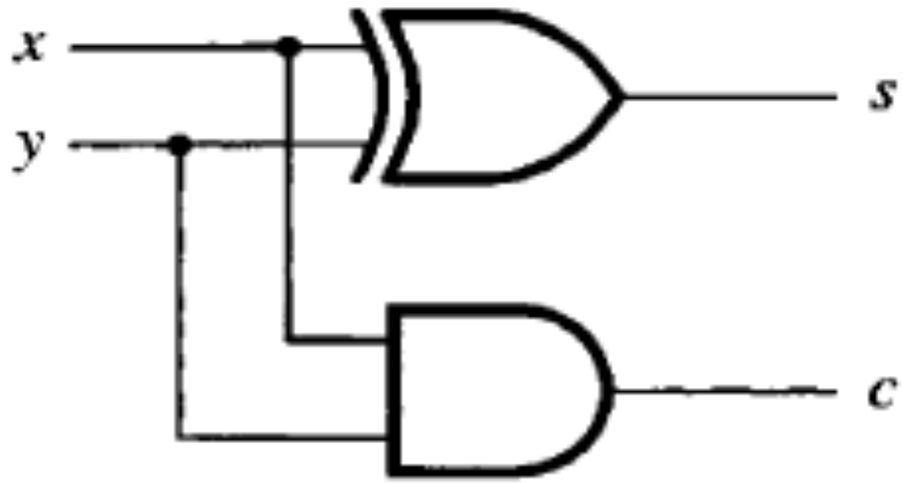
Unsigned Arithmetic - Half Adder

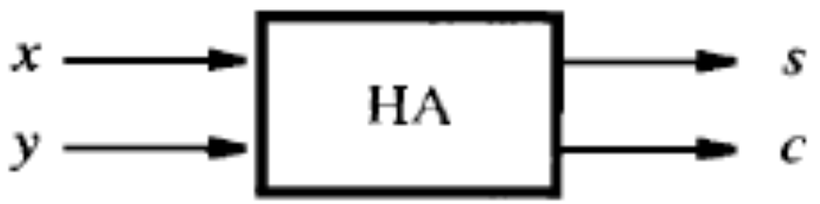
This circuit, which implements the addition of only two bits, is called a half-adder.

x	0	0	1	1
$+\ y$	$+\ 0$	$+\ 1$	$+\ 0$	$+\ 1$
$c\ s$	0 0	0 1	0 1	1 0

Carry — Sum

x	y	Carry c	Sum s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0





s = xy' + x'y

c = xy

Unsigned Arithmetic - Multi bit

By Hand Method

Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

How to design a digital circuit for this ???

Unsigned Arithmetic - Multi bit

Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

- Regular truth table approach would be impractical

- The required truth table would have 10 input variables,S for each number X and Y. It would have 2^{10} = 1024 rows!

- A better approach is to consider the addition of each pair of bits, Xi and Yi, separately.

Unsigned Arithmetic - Multi bit

Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

- For bit position 0, there is no carry-in, and hence the addition is the same as Half Adder

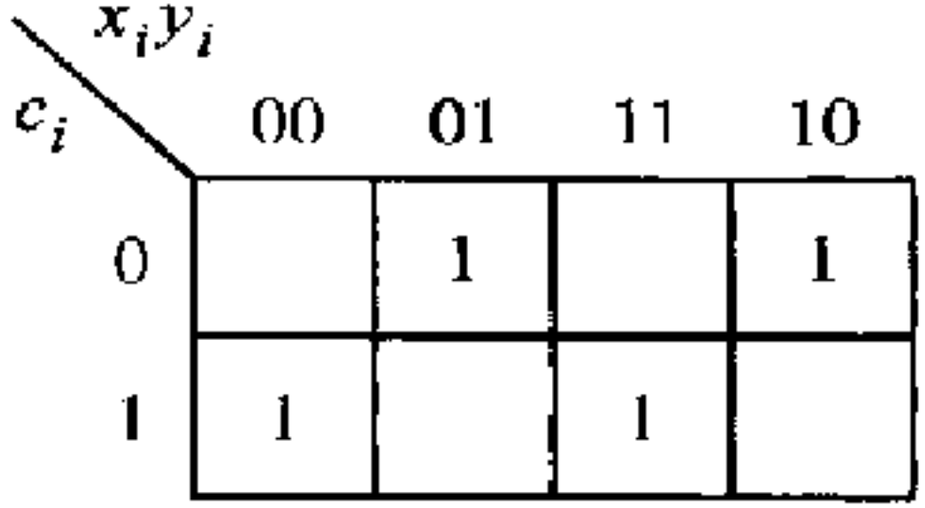
- For each other bit position i. the addition involves bits X_i and Y_i, and a carry-in C_i

- This is a three variable function, for which truth table can be derived.

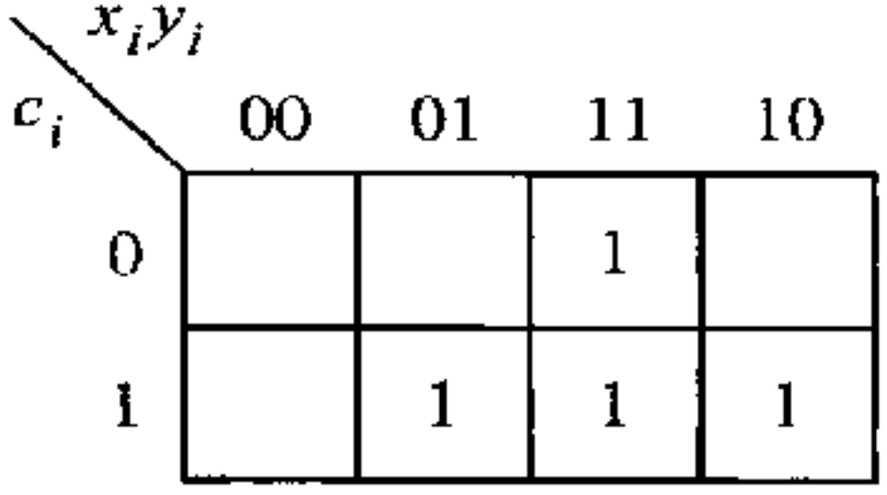
Unsigned Arithmetic - Full Adder

This circuit, which implements the addition of only three bits, is called a Full-adder.

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

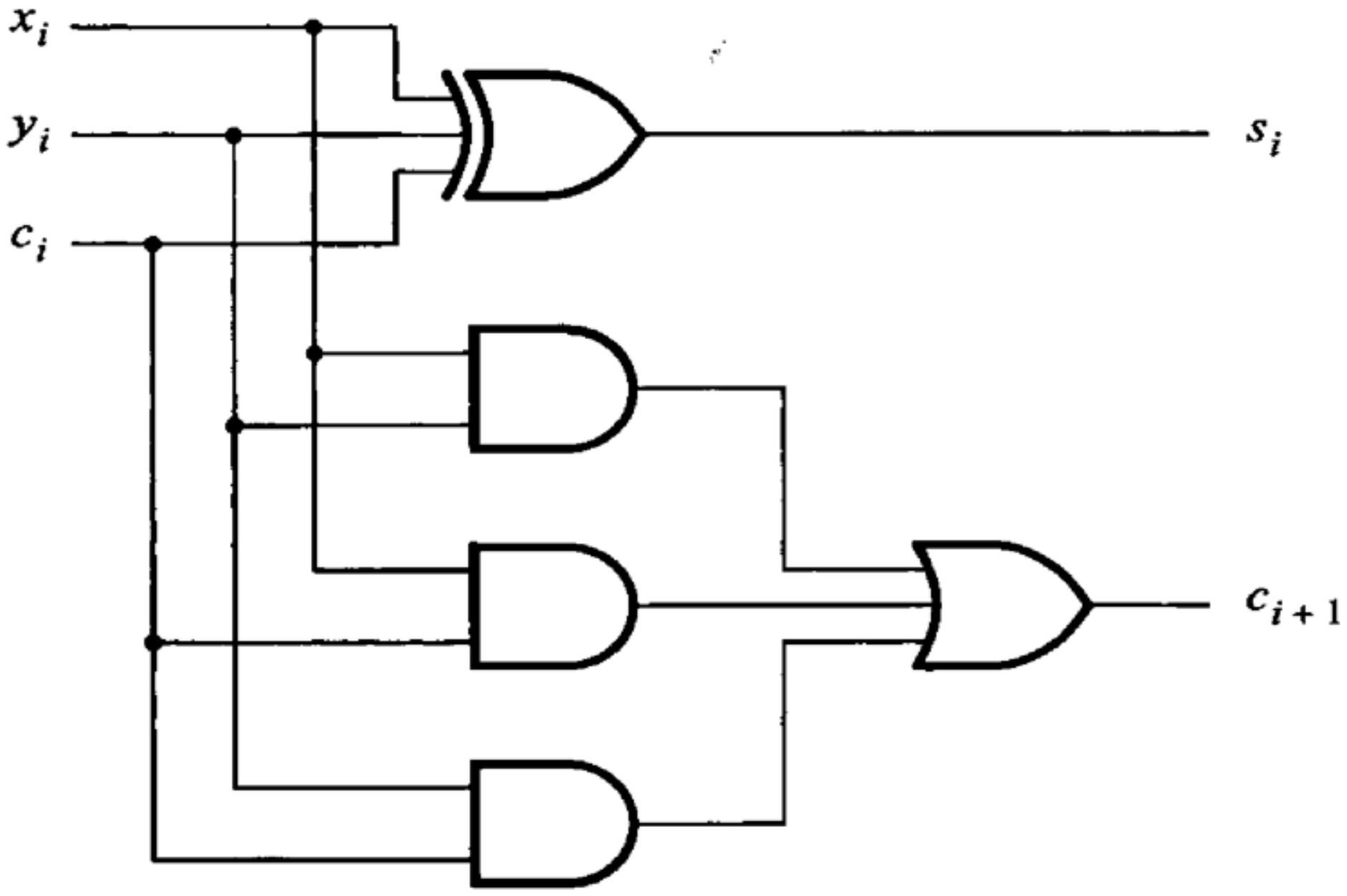


$$s_i = x_i \oplus y_i \oplus c_i$$

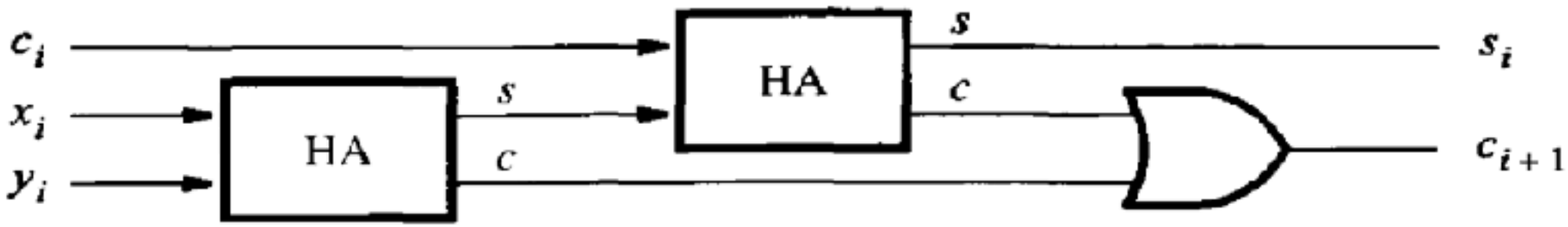


$$c_{i+1} = x_iy_i + x_ic_i + y_ic_i$$

Unsigned Arithmetic - Full Adder

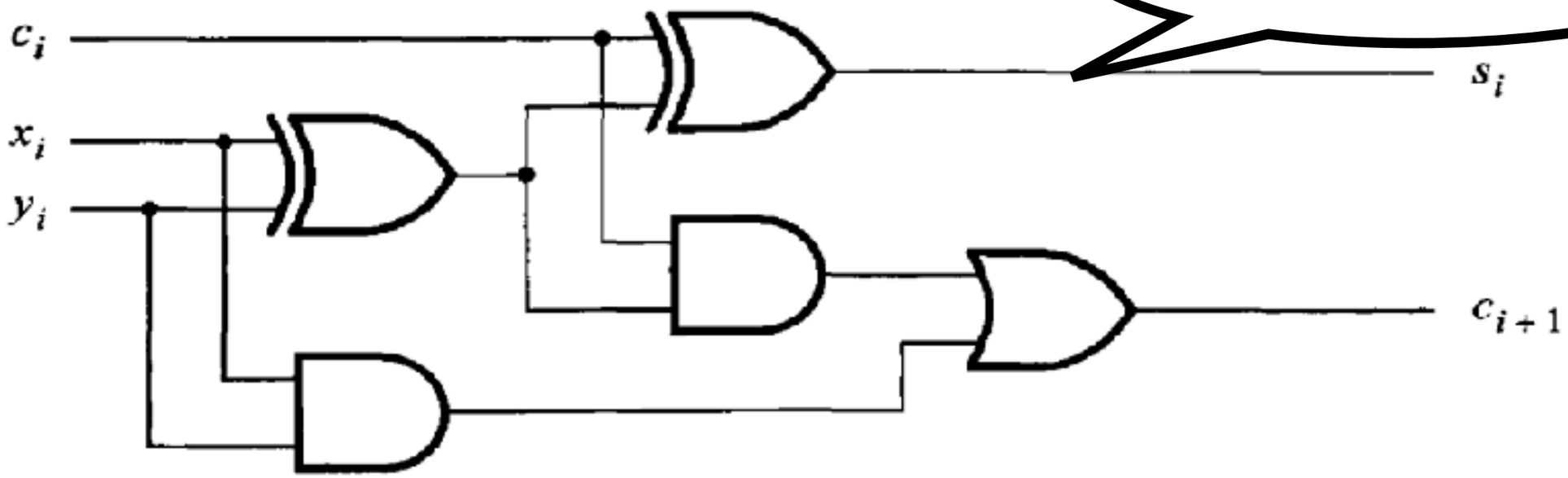


Unsigned Arithmetic - Decomposed Full Adder



(a) Block diagram

2 Half Adder = 1 Full Adder



(b) Detailed diagram

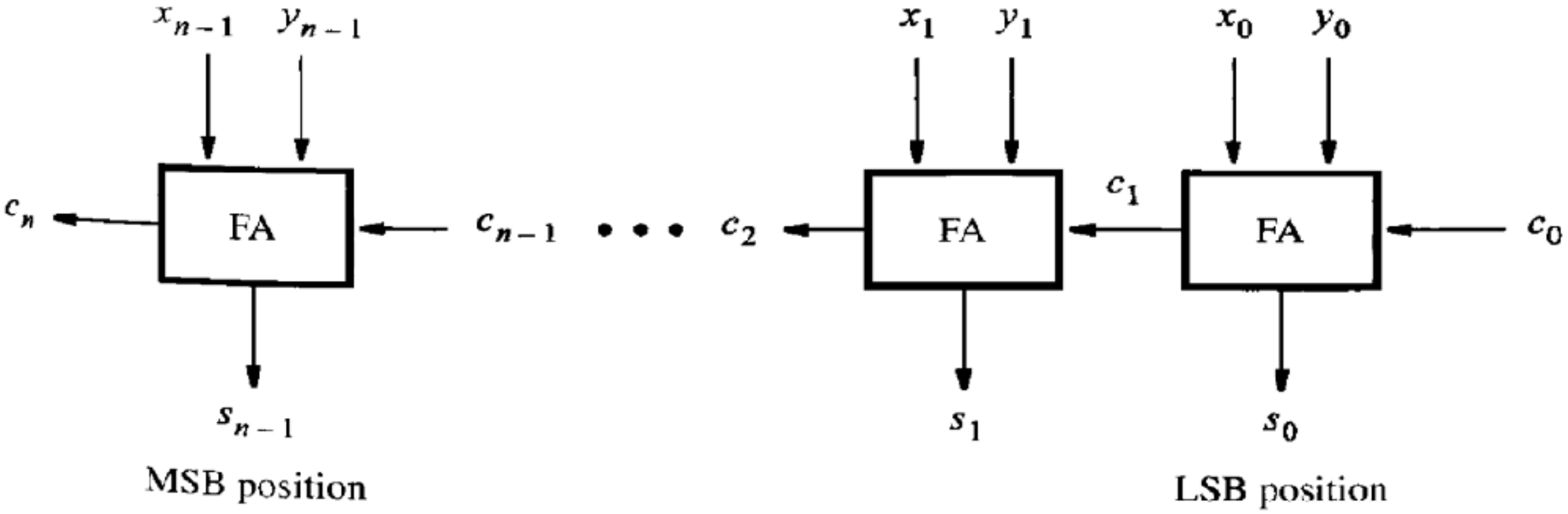
Unsigned Arithmetic - Ripple Carry Adder

- To perform addition by hand, we start from the least-significant digit and add pairs of digits, progressing to the most-significant digit

- If a carry is produced in position i, then this carry is added to the operands in position i + 1

- The same arrangement can be used in a logic circuit that performs addition.

Unsigned Arithmetic - Ripple Carry Adder



Unsigned Arithmetic - Ripple Carry Adder

- When the operands X and Y are applied as inputs to the adder, it takes some time before the output sum, S, is valid.

- Each full-adder introduces a certain delay before its S_i and C_{i+1} outputs are valid Let this delay be denoted as $\triangle t$

- Thus the carry-out from the first stage, C_1 arrives at the second stage $\triangle t$ after the application of the x_0 and y_0 inputs. The carry-out from the second stage, C_2, arrives at the third stage with a $2\triangle t$ delay, and so on

- The signal C_{n-1} is valid after a delay of $(n - 1)\triangle t$, which means that the complete sum is available after a delay of $n\triangle t$.

Unsigned Arithmetic - Design Example

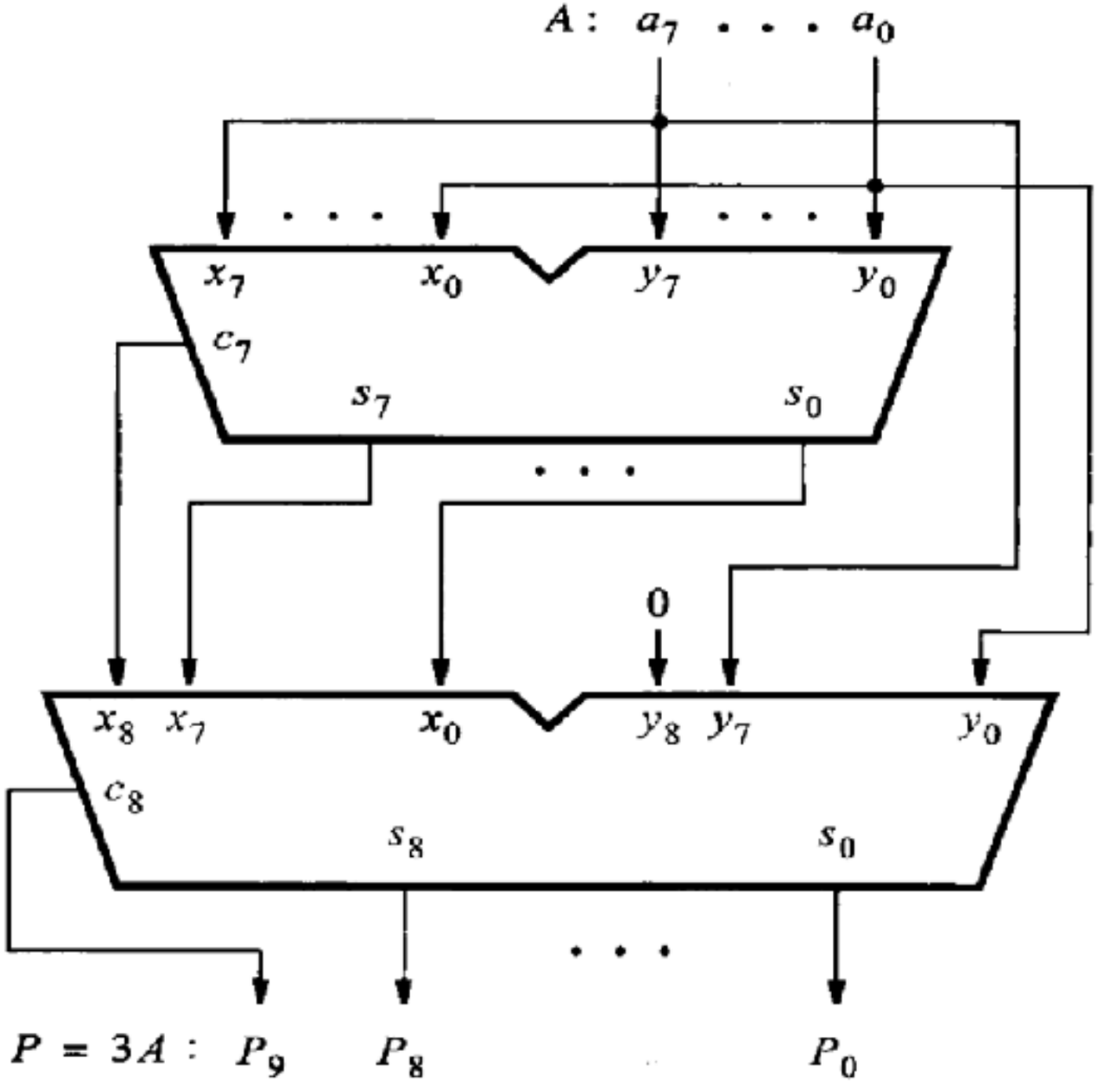
- Design a multiplier that multiplies an eight bit unsigned number by 3.

- P = 3A

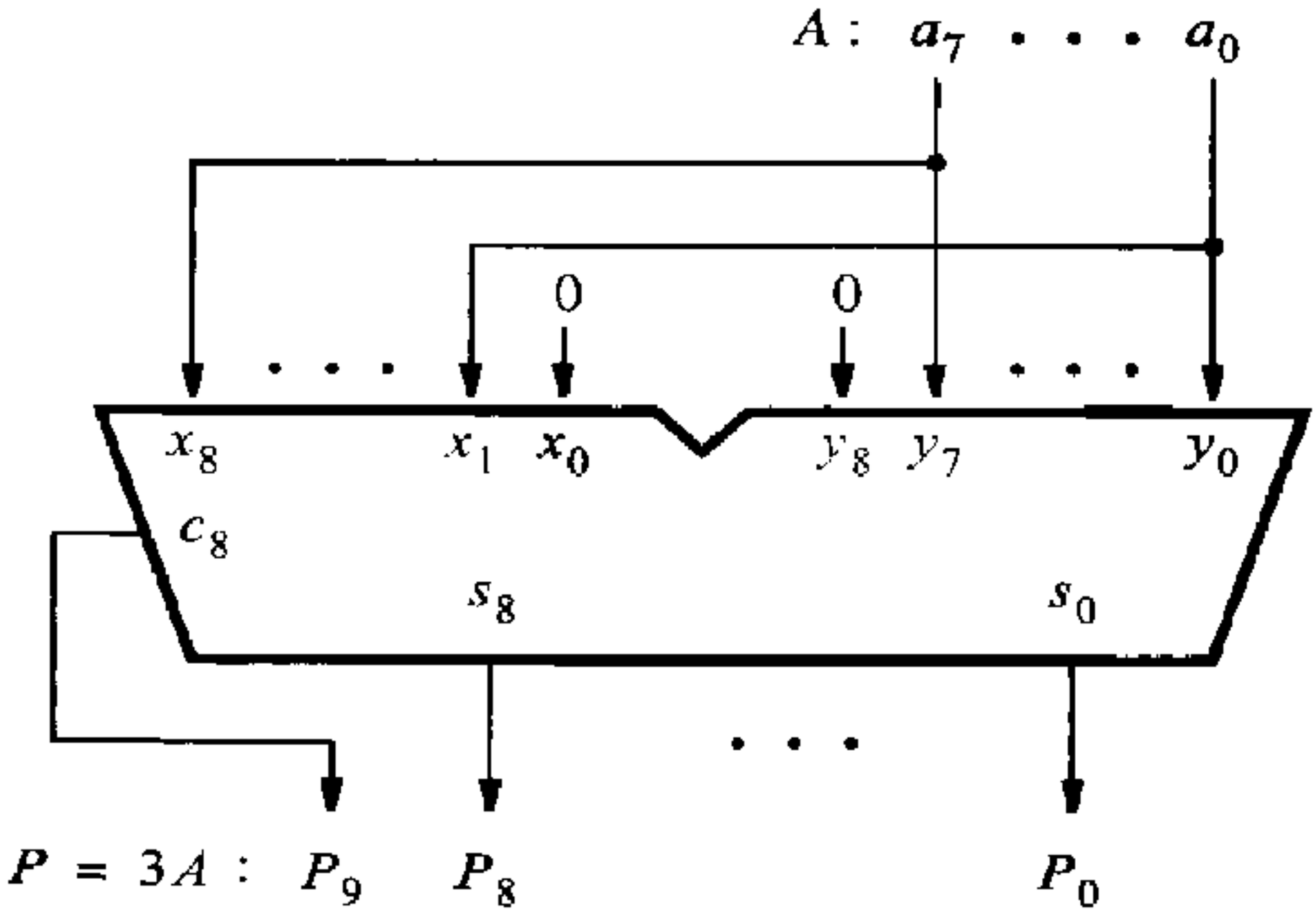
- A = a_7,a_6,…a_0

- P = p_7, p_6, …p_0

Unsigned Arithmetic - Design Example



Unsigned Arithmetic - Design Example



Unsigned Arithmetic - Half and Full Subtractor

- Design a Half Subtractor circuit with two inputs X, Y and two outputs D (Difference), B(Borrow)

  - Hint:

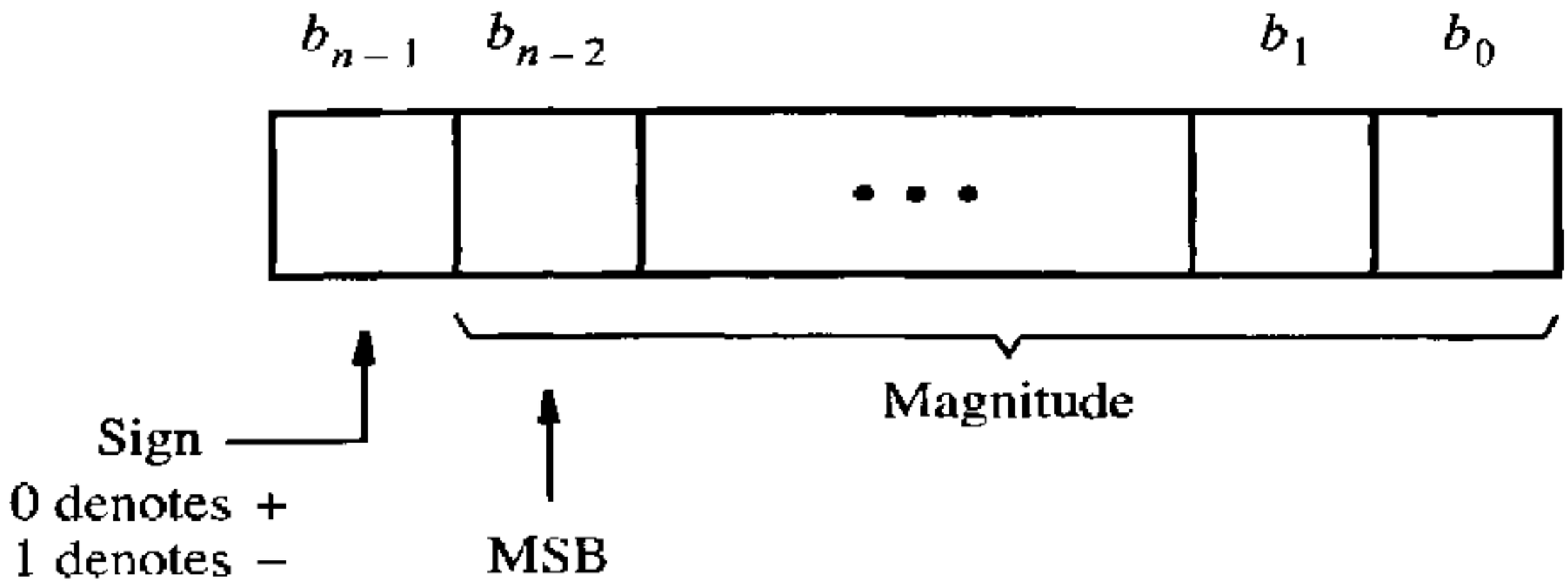
HS - Truth Table			
X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- Design a Full Subtractor circuit with three inputs X, Y, Z and two outputs D (Difference), B(Borrow)

Signed Number Representation

- Positive numbers are represented using the positional number representation as explained in the previous section.
- Negative numbers can be represented in three different ways:
  - Sign-and-magnitude
  - 1 's complement
  - 2's complement.

Signed Number Representation- Sign Magnitude Representation



Example:
+5 = 0101
-5 = 1101

Drawbacks
- **Range**
- **Circuit complexity**

Signed Number Representation - 1's Complement

- In a complementary number system, the negative numbers are defined according to a subtraction operation involving positive numbers.

- In the 1 s complement scheme. an n-bit negative number, K, is obtained by subtracting its equivalent positive number, P, from 2^n-1

  - $K = (2^n-1)-P$

Example:
$-5 = (2^4-1)-0101 = 1111-0101 = 1010$
$-3 = (2^4-1)-0011 = 1111-0011 = 1100$

Simply it is complement of its positive number

Signed Number Representation - 2's Complement

- In the 2's complement scheme, a negative number, K, is obtained by subtracting its equivalent positive number, P, from 2^n

  - $K = 2^n - P$

Example:
$-5 = 2^4 - 0101 = 10000 - 0101 = 1011$
$-3 = 2^4 - 0011 = 10000 - 0011 = 1101$

Simply it is 1's complement plus one

Signed Number Representation

$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
0000	+0	+0	+0
1000	−0	−7	−8
1001	−1	−6	−7
1010	−2	−5	−6
1011	−3	−4	−5
1100	−4	−3	−4
1101	−5	−2	−3
1110	−6	−1	−2
1111	−7	−0	−1

Signed Arithmetic - Sign Magnitude

- If both operands have the same sign, then the addition of sign-and-magnitude numbers is simple. The magnitudes are added, and the resulting sum is given the sign of the operands.

- However, if the operands have opposite signs, the task becomes more complicated. Then it is necessary to subtract the smaller number from the larger one.

- This means that logic circuits that compare and subtract numbers are also needed.

- For this reason, the sign-and-magnitude representation is not used in computers

Signed Arithmetic - 1's Complement Addition

- An obvious advantage of the 1 's complement representation is that a negative number is generated simply by complementing all bits of the corresponding positive number

(+5)	0101	(–5)	1010
+ (+2)	+ 0010	+ (+2)	+ 0010
(+7)	0111	(–3)	1100

(+5)	0101	(–5)	1010
+ (–2)	+ 1101	+ (–2)	+ 1101
(+3)	10010	(–7)	10111
	1		1
	0011		1000

Adding the extra carry is an overhead

Signed Arithmetic - 2's Complement Addition

(+5)	0101	(–5)	1011
+ (+2)	+ 0010	+ (+2)	+ 0010
(+7)	0111	(–3)	1101

(+5)	0101	(–5)	1011
+ (–2)	+ 1110	+ (–2)	+ 1110
(+3)	10011	(–7)	11001
	ignore		ignore

No extra addition of carry is required, hence this is more suitable for addition operation

Signed Arithmetic - 2's Complement Subtraction

$$\begin{array}{r} (+5) \\ -(+2) \\ \hline (+3) \end{array} \qquad \begin{array}{r} 0101 \\ -\ 0010 \\ \hline \end{array} \qquad \Rightarrow \qquad \begin{array}{r} 0101 \\ +\ 1110 \\ \hline 10011 \end{array}$$

ignore

$$\begin{array}{r} (-5) \\ -(+2) \\ \hline (-7) \end{array} \qquad \begin{array}{r} 1011 \\ -\ 0010 \\ \hline \end{array} \qquad \Rightarrow \qquad \begin{array}{r} 1011 \\ +\ 1110 \\ \hline 11001 \end{array}$$

ignore

$$\begin{array}{r} (+5) \\ -(-2) \\ \hline (+7) \end{array} \qquad \begin{array}{r} 0101 \\ -\ 1110 \\ \hline \end{array} \qquad \Rightarrow \qquad \begin{array}{r} 0101 \\ +\ 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ -(-2) \\ \hline (-3) \end{array} \qquad \begin{array}{r} 1011 \\ -\ 1110 \\ \hline \end{array} \qquad \Rightarrow \qquad \begin{array}{r} 1011 \\ +\ 0010 \\ \hline 1101 \end{array}$$

Hence, Subtraction can be done by adder itself

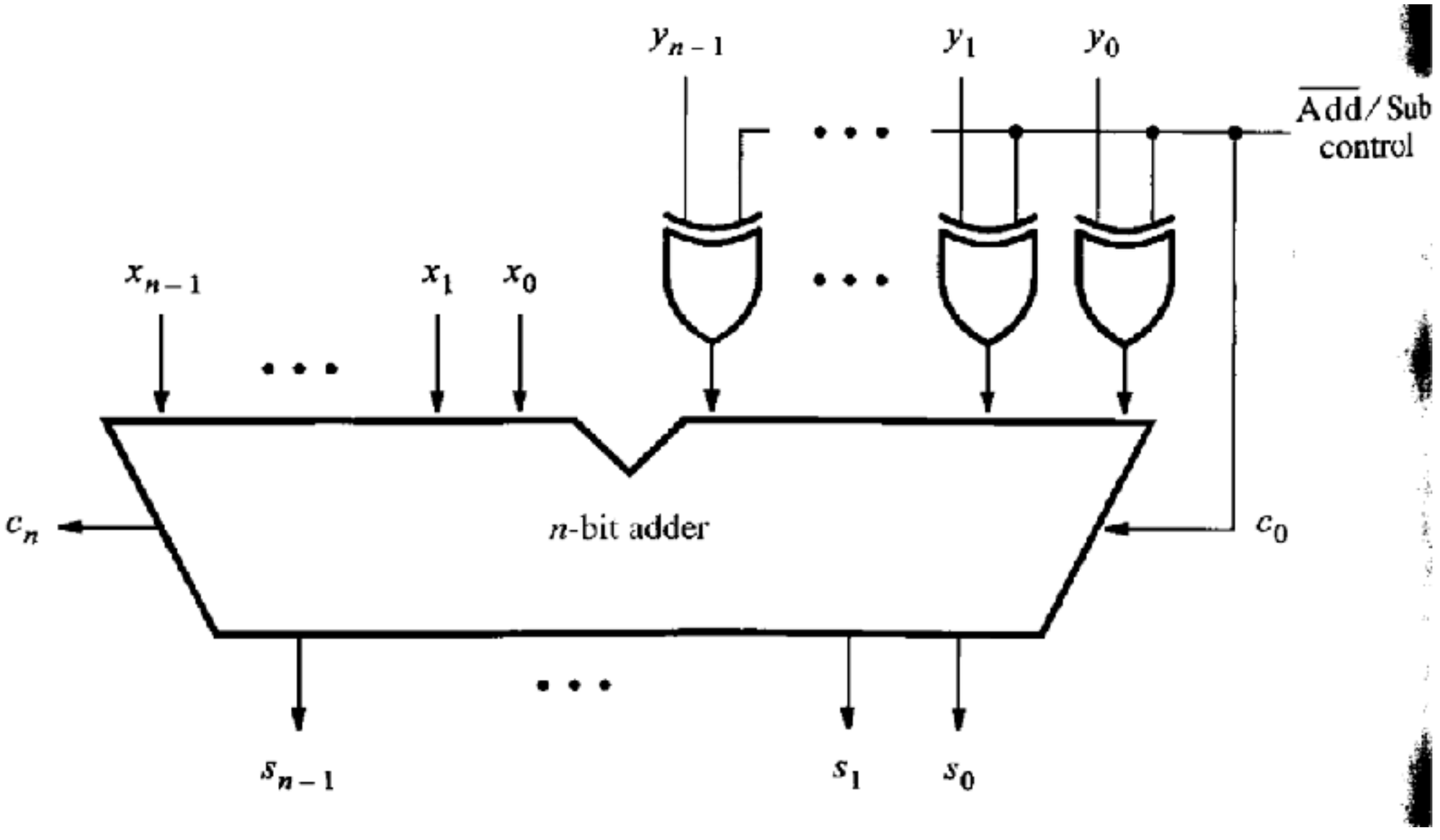
Signed Arithmetic - 2's Complement

- Perform the following
  - $(+10)_{10} + (-12)_{10}$
  - $(+10)_{10} - (+15)_{10}$

+10	2's comp form	01010
-12	2's comp form	10100
-2	2's comp form	11110

+10	2's comp form	01010
-(+15)	2's comp form	10001
-5	2's comp form	11011

Signed Arithmetic - Adder and Subtractor Unit



Arithmetic Overflow

- The result of addition or subtraction is supposed to fit within the significant bits used to represent the numbers.

- If n bits are used to represent signed numbers, then the result must be in the range -2^{n-1} to $2^{n-1} - 1$.

- If the result does not fit in this range, then we say that arithmetic overflow has occurred.

- To ensure the correct operation of an arithmetic circuit, it is important to be able to detect the occurrence of overflow.

Arithmetic Overflow

(+ 7)	0 1 1 1
+ (+ 2)	+ 0 0 1 0
(+ 9)	1 0 0 1

$c_4 = 0$
$c_3 = 1$

(–7)	1 0 0 1
+ (+ 2)	+ 0 0 1 0
(–5)	1 0 1 1

$c_4 = 0$
$c_3 = 0$

(+ 7)	0 1 1 1
+ (–2)	+ 1 1 1 0
(+ 5)	1 0 1 0 1

$c_4 = 1$
$c_3 = 1$

(–7)	1 0 0 1
+ (–2)	+ 1 1 1 0
(–9)	1 0 1 1 1

$c_4 = 1$
$c_3 = 0$

$$\text{Overflow} = c_3\bar{c}_4 + \bar{c}_3 c_4$$
$$= c_3 \oplus c_4$$

$$\text{Overflow} = c_{n-1} \oplus c_n$$

Fast Adders

- The performance of a large digital system is dependent on the speed of circuits that form its various functional units.
- Obviously, better performance can be achieved using faster circuits.
- This can be accomplished by using superior (usually newer) technology in which the delays in basic gates are reduced.
- But it can also be accomplished by changing the overall structure of a functional unit, which may lead to even more impressive improvement.
- To reduce the delay caused by the effect of carry propagation through the ripple-carry adder, we can attempt to evaluate quickly for each stage whether the carry-in from the previous stage will have a value 0 or I.
- If a correct evaluation can be made in a relatively short time, then the performance of the complete adder wi II be improved.

Fast Adders - Carry Look Ahead Adder

- Based on Full adder circuit the carry-out function for stage i can be realized as

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

If we factor this expression as

$$c_{i+1} = x_i y_i + (x_i + y_i) c_i$$

then it can be written as

$$c_{i+1} = g_i + p_i c_i$$

where

$$g_i = x_i y_i$$
$$p_i = x_i + y_i$$

Fast Adders - Carry Look Ahead Adder

- Expanding in term of i-1 gives

$$c_{i+1} = g_i + p_i(g_{i-1} + p_{i-1}c_{i-1})$$
$$= g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1}$$

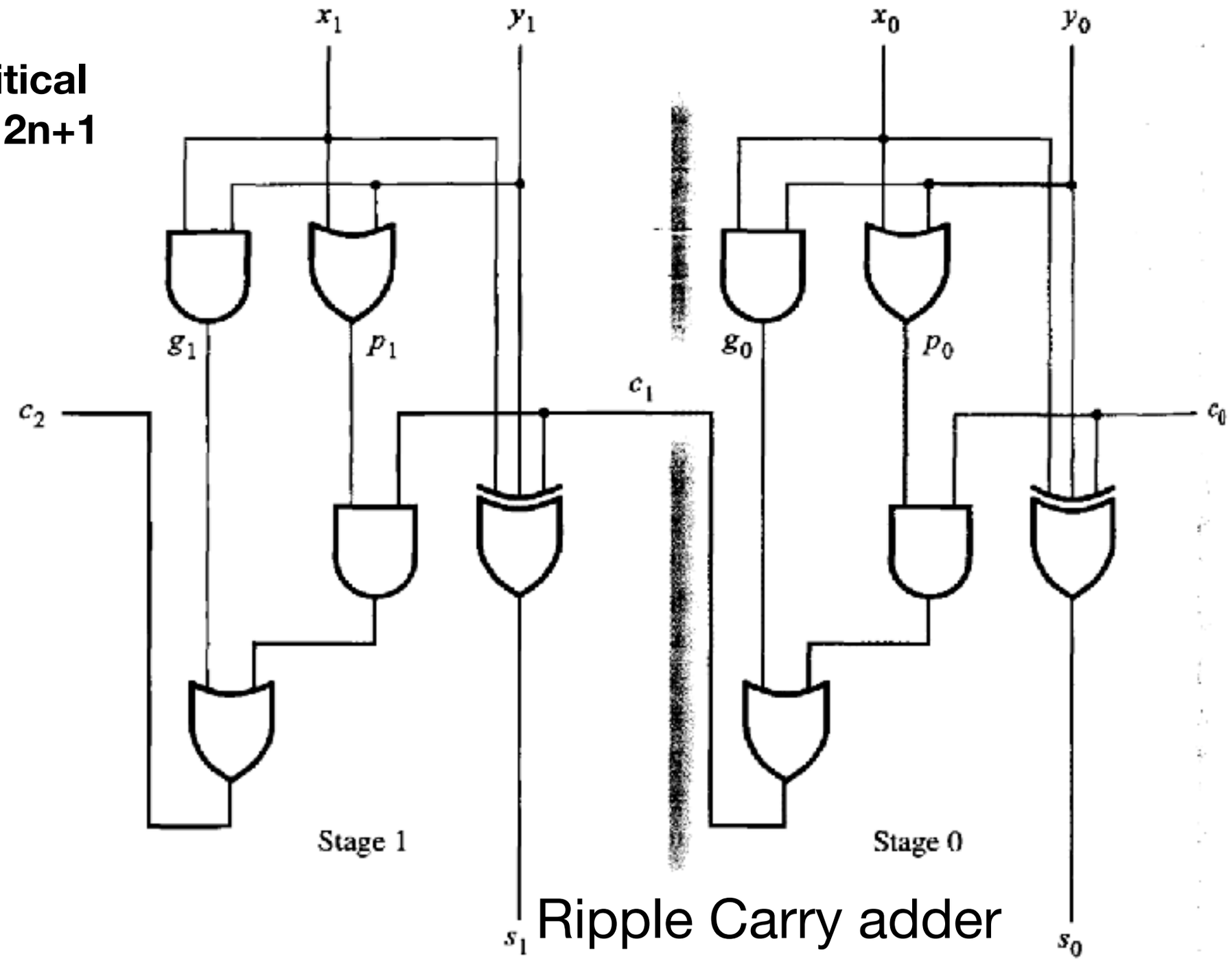
The same expansion for other stages, ending with stage 0, gives

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$

- This expression represents a two-level AND-OR circuit in which C_{i+1} is evaluated very quickly. An adder based on this expression is called a carry-lookahead adder.

Fast Adders - Carry Look Ahead Adder

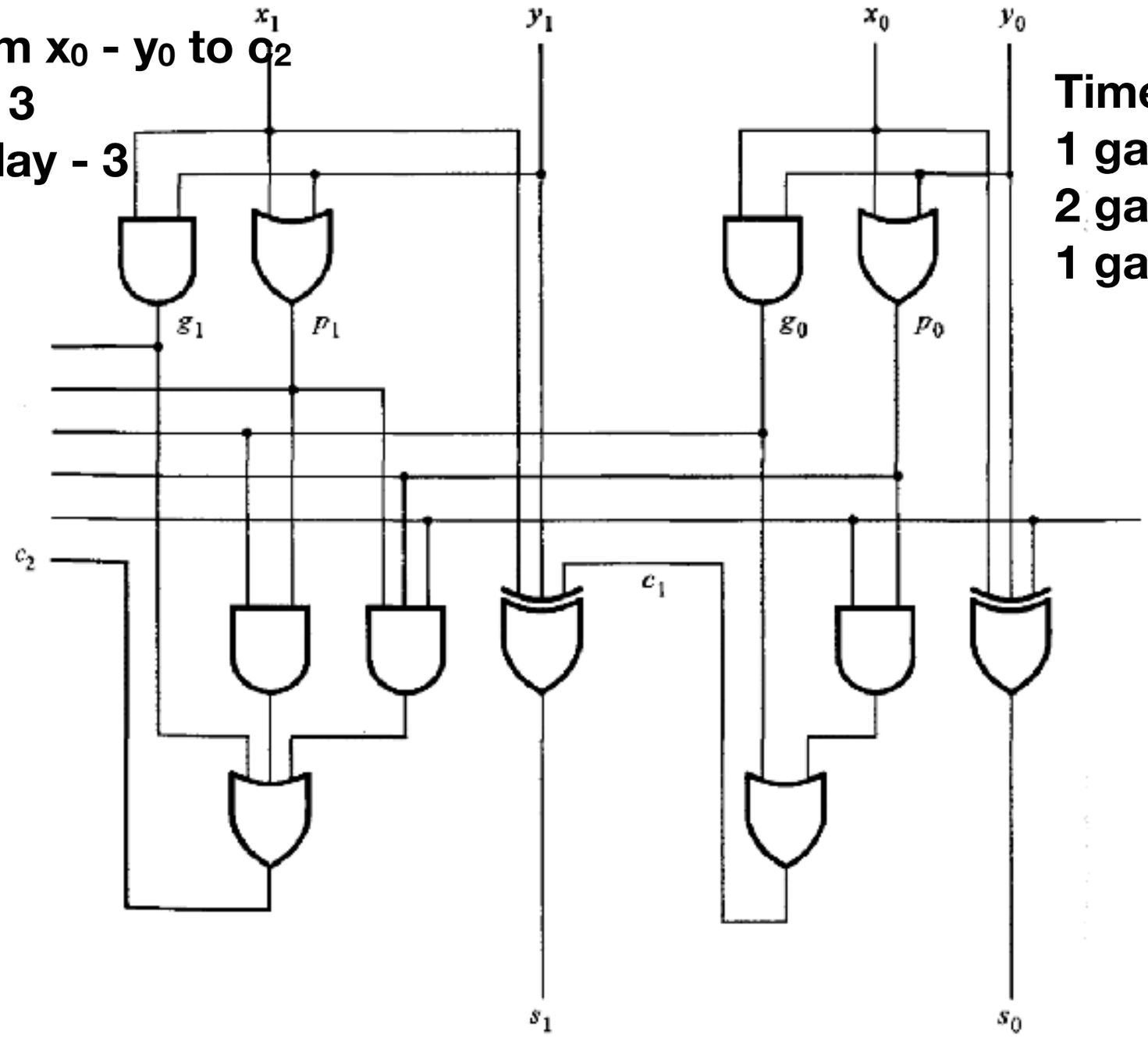
Critical Path : From x_0 - y_0 to c_2
For n bit critical path will be 2n+1



Ripple Carry adder

Fast Adders - Carry Look Ahead Adder

Critical Path : From x_0 - y_0 to c_2
Gate Delay for c_2 - 3
For n bits gate delay - 3



Time Delay
1 gate delay - P_i & G_i
2 gate delay - C
1 gate delay - S

Carry look Ahead Adder

Thank You !!!